

RC Car Track Racing Using Advanced Control Strategies

Wissam Tedros Yasmine Al-Ek Fouad Hassanieh

Department of Electrical and Computer Engineering

American University of Beirut

Beirut, Lebanon

Emails: {wct00, yna11, fwh04}@mail.aub.edu

Abstract—This document presents the design and ongoing development of an autonomous RC racing platform integrating adaptive control and reinforcement learning (RL) for trajectory tracking. The vehicle, built around a dual BLDC motor drivetrain and a front steering servo, is modeled using a 3-DOF dynamic bicycle model with linearized tire forces. A hierarchical Model Reference Adaptive Control (MRAC) architecture is proposed, comprising a slow outer loop that compensates for battery discharge effects on propulsion and a fast inner loop that adapts to grip variation through online cornering stiffness estimation via Recursive Least Squares (RLS). Torque vectoring through the independent rear motors is explicitly incorporated as a second yaw control channel. An RL module is designed to operate at the local planning level, learning steering corrections within the waypoint-based pipeline without replacing the classical control stack. The controllers will be validated in a modified Gazebo simulation environment featuring a ramp and tunnel, and the Hardware deployment on a custom NXP-based platform is in progress.

Index Terms—Adaptive, Reinforcement Learning, RC Car, Line Follower.

I. INTRODUCTION

Autonomous racing is a demanding benchmark problem in robotics and control, requiring high-speed trajectory tracking, rapid adaptation to uncertainty, and performance-oriented decision making. Small-scale racing platforms are particularly sensitive to modeling errors, actuator nonlinearities, and changing surface conditions, making them suitable testbeds for advanced control strategies.

In this project, we investigate the integration of adaptive control and reinforcement learning (RL) within a waypoint-based autonomous racing pipeline. Adaptive control will address modeling uncertainty and parameter variation, while RL will be used to enhance local performance without replacing the classical control structure. The proposed framework will be validated in Gazebo simulation and deployed on a custom-designed RC vehicle, enabling a complete simulation-to-real implementation aligned with course objectives.

II. MOTIVATION

This project aims to apply the adaptive control and reinforcement learning (RL) techniques studied in the course to a fully custom autonomous RC racing platform. The vehicle will be designed and partially 3D-printed from scratch, enabling co-design of the mechanical structure and the control architecture.

Adaptive control will be used to ensure robust trajectory tracking under modeling uncertainties inherent to a low-cost, custom-built platform. Reinforcement learning will be integrated within a classical waypoint-based pipeline to improve local decision-making and racing performance, rather than replacing the entire control stack. The controllers will first be validated in simulation using Gazebo and then deployed on the physical platform, ensuring a structured simulation-to-real workflow.

Finally, the project contributes to our University's robotics team competing in the NXP Cup autonomous RC competition, which has faced robustness and integration challenges in previous years. By developing a custom hardware platform together with a structured adaptive and RL-based control framework, we aim to produce a technically rigorous course project with direct competitive relevance.

III. LITERATURE REVIEW

Autonomous racing has become a strong benchmark problem in robotics and control, as it requires a vehicle to operate close to its physical limits while still ensuring safe navigation, rapid decision-making, and reliable performance in uncertain conditions. In a typical racing stack, a global planner generates a reference racing line, which is followed by a local planner and a low-level controller responsible for executing steering and speed commands in real time. However, a key difficulty arises because classical model-based local planning methods often rely on either continuously updated obstacle maps or highly accurate system models, which may be unrealistic for fast onboard systems. As a result, Reinforcement Learning (RL) has gained attention as a learning-centric alternative that can produce high-performance policies directly from interaction with the environment, without the need to explicitly maintain a map and lessening dependence on perfectly known dynamics.

Evans et al. [1] present a hybrid autonomous racing framework that utilizes a classical waypoint-based navigation pipeline while integrating reinforcement learning at the local planning level. They combine a pure-pursuit path follower with an RL agent that runs in parallel. The path follower tracks the global waypoint sequence and produces a baseline steering reference, while the RL agent modifies this reference online to improve local behavior. Hence, the RL policy does not replace

the full planning stack; instead, it learns a steering correction policy from compact local state and perception inputs. In addition, the reward design encourages behavior that remains close to the generated path and penalizes deviations away from it. Then, the framework preserves a clear link between global waypoint guidance and real-time local adaptation, which is valuable in racing, where conditions can change quickly.

Another foundational issue is reward design, since it strongly determines what the RL agent learns. In [2], where also Evans et al. organize the problem around three reward methodologies: position-based rewards, velocity-based rewards, and action-based rewards. They discussed how each reward method encodes different racing objectives and performed a simulated comparison in an F1/10 setting, which allowed a direct evaluation of how reward choice influences lap time and driving behavior, not only lap completion. A key contribution is their proposal to reward the agent relative to a pre-calculated minimum-curvature path rather than only the track centerline. This is an important shift since it ties reward design more directly to the racing objective, not just lane-following accuracy.

A key issue in RL for autonomous racing is choosing a learning algorithm that balances performance, stability, and real-time feasibility under tight onboard compute constraints. [3] discusses this challenge by comparing both model-free and model-based approaches in high-speed racing tasks. They explain that Proximal Policy Optimization (PPO) offers stable policy updates through its clipped objective, which makes training more reliable, but it is generally less data efficient. In contrast, Soft Actor-Critic (SAC) is more sample efficient and often more robust in continuous control, though its performance can be sensitive to reward scaling and hyperparameter tuning. To study these tradeoffs, the authors benchmark PPO and SAC against a lightweight world-model framework designed for racing, called LiDAR Dreamer. Their method reduces model complexity through efficient LiDAR encoding and a lighter recurrent state space design, making it more suitable for embedded deployment. The results show that PPO and SAC perform strongly on simpler tracks, where direct policy learning is sufficient, but world-model methods become more competitive as track complexity increases. This suggests that algorithm choice in racing should depend not only on raw performance, but also on track difficulty, sample efficiency, and computational budget.

The development of a robust adaptive control architecture for an autonomous racecar requires integrating high-level trajectory optimization with precise low-level motor regulation, beginning with the selection of an appropriate underlying vehicle model. Kong et al. [4] conducted a comparative analysis between kinematic and dynamic bicycle models to evaluate their forecast error statistics using experimental data. Their study revealed the counterintuitive finding that a simple kinematic model, when discretized with a longer sampling time, can achieve open-loop prediction accuracy comparable to more complex dynamic models. They demonstrate that a kinematic model discretized at 200 ms yields comparable

open-loop prediction errors to dynamic linear tire models (e.g., 0.33 m vs. 0.31 m at 800 ms), attributed to truncation errors mimicking slip effects. Furthermore, they highlighted that dynamic models relying on linear tire approximations often become singular at low vehicle speeds, demonstrating that computationally efficient kinematic models are highly reliable for Model Predictive Control (MPC) across varying racing scenarios. However, to make such kinematic or dynamic controllers truly adaptive in high-speed racing, the system must accurately account for the rapidly changing tire-road interface. Addressing this, Kim et al. [4] proposed a real-time friction coefficient estimation method that relies solely on onboard six-degrees-of-freedom (6-DoF) acceleration measurements rather than complex tire force sensors. By applying a recursive least squares (RLS) algorithm to derived longitudinal, lateral, and normal accelerations, the system can dynamically track changing friction conditions on the fly. Experimental validation during emergency braking maneuvers demonstrated that this acceleration-based method provides robust, real-time friction tracking, which is essential for adaptive controllers that must quickly adjust to varying track surfaces. Lin et al. [5] addressed this critical actuation challenge by proposing a robust nonlinear speed control scheme for brushless DC (BLDC) motors based on a Model Reference Adaptive Backstepping (MRAB) approach, eliminating the need for traditional PI loops by directly providing PWM reference voltages. Their approach models brushless DC motor dynamics under two-phase conduction and treats parametric uncertainties, such as resistance and friction, and disturbances as mismatched terms. After that, adaptive laws are used with a linear reference model to achieve the desired performance. Simulations confirmed that this approach provided better tracking, greater robustness to load variations, and faster convergence compared to conventional PI methods. However, it should be noted that the motors discussed in the previous paper are limited to brushless DC motors with trapezoidal back electromotive force. Expanding on these advanced motor regulation techniques, Melkia et al. [6] explored an improved backstepping control strategy tailored for the speed regulation of Permanent Magnet Synchronous Motors (PMSMs). Their approach decomposes the complex nonlinear motor system into multiple hierarchical stages, using Lyapunov stability principles to ensure global system stability at each sequential step. This systematic error regulation allows the motor to dynamically adapt to parameter variations and external perturbations without the chattering effects commonly seen in sliding mode control methodologies. Ultimately, their comparative simulations demonstrated that the backstepping controller reduced settling time by 45% and improved tracking accuracy by 30% relative to conventional PI controllers, proving highly resilient against sudden torque disturbances.

Vehicle modeling in autonomous racing often begins with reduced representations that balance physical meaning and control tractability. In [7], the vehicle is modeled as a single-track bicycle system with separate earth, body, wheel, and axle frames. This framework allows motion and force relationships

to be written through clear coordinate transformations. The same work derives the vehicle dynamics using the Euler-Lagrange method, with generalized coordinates for position and yaw, and it expresses tire forces through cornering stiffness and wheel frame velocities. It also reformulates the equations in body frame variables and develops both nonlinear and linearized state space models, which makes the formulation suitable for control-oriented analysis. Kong et al. in [8] extend this discussion by comparing kinematic and dynamic bicycle models for autonomous driving. They show that the kinematic model captures motion through wheelbase geometry and steering, whereas the dynamic model includes lateral velocity, yaw dynamics, and linear tire forces through slip angles and cornering stiffness. They also show that discretization is an important part of the modeling choice, since a kinematic model sampled at a slower rate can approximate some understeer behavior while still supporting MPC with lower computational cost.

Although classical and adaptive control strategies, provide a mathematically guaranteed foundation for robust trajectory tracking and speed regulation in custom RC racing platforms, their integration with Reinforcement Learning offers a hybrid approach to learn optimal, adaptive corrections that surpass purely model-based limits in dynamic racing conditions.

IV. VEHICLE MODELING

The system is assumed to be the kinematic and dynamic bicycle model, on a 2D plane, where the longitudinal, lateral and yaw dynamics of the car are drafted (following [9], [10]), then merged with the individual model of each subsystem such as the brushless DC motors, the servo steering system, and the observers based on the camera, 6-DOF IMU and the 2 incremental encoders on each motor.

A. Kinematic Bicycle Model

The kinematic bicycle model shown in 1 assumes that the tire does not slip lateral. The equations of motion in the inertial frame are given by

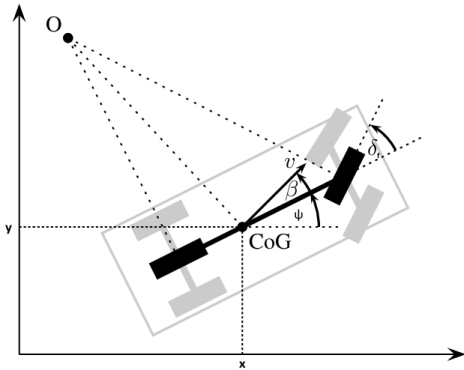


Fig. 1: Kinematic bicycle modeling of ackermann steering car

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\psi} \\ \dot{v} \end{bmatrix} = \begin{bmatrix} v \cos(\psi + \beta) \\ v \sin(\psi + \beta) \\ \frac{v}{l_r} \sin(\beta) \\ a \end{bmatrix}, \quad \beta = \tan^{-1} \left(\frac{l_r}{l_f + l_r} \tan(\delta_f) \right) \quad (1)$$

where $\begin{cases} x, y & \text{inertial position coordinates (m)} \\ \psi & \text{yaw angle (rad)} \\ v & \text{longitudinal speed (m/s)} \\ \beta & \text{side-slip angle at CoM (rad)} \\ l_f, l_r & \text{distances from CoM to axles (m)} \\ a & \text{longitudinal acceleration input (m/s}^2\text{)} \\ \delta_f & \text{front steering angle (rad)} \end{cases}$

B. Dynamic Bicycle and tire Model

1) *Dynamic bicycle model*: The dynamic bicycle model accounts for lateral tire forces and slip dynamics. The equations are

$$\begin{bmatrix} \ddot{x} \\ \ddot{y} \\ \ddot{\psi} \\ \dot{X} \\ \dot{Y} \end{bmatrix} = \begin{bmatrix} \dot{\psi} \dot{y} + a_x \\ -\dot{\psi} \dot{x} + \frac{2}{m} (F_{c,f} \cos \delta_f + F_{c,r}) \\ \frac{1}{I_z} (l_f F_{c,f} - l_r F_{c,r}) \\ \dot{x} \cos \psi - \dot{y} \sin \psi \\ \dot{x} \sin \psi + \dot{y} \cos \psi \end{bmatrix} \quad (2)$$

where $\begin{cases} \dot{x}, \dot{y} & \text{longitudinal and lateral velocities in body frame (m/s)} \\ \ddot{x}, \ddot{y} & \text{longitudinal and lateral accelerations in body frame (m/s}^2\text{)} \\ \dot{\psi} & \text{yaw rate (rad/s)} \\ \ddot{\psi} & \text{yaw acceleration (rad/s}^2\text{)} \\ a_x & \text{longitudinal acceleration in body frame (m/s}^2\text{)} \\ m & \text{vehicle mass (kg)} \\ F_{c,f}, F_{c,r} & \text{front and rear lateral tire forces (N)} \\ I_z & \text{yaw moment of inertia (kg} \cdot \text{m}^2\text{)} \\ X, Y & \text{global position coordinates in inertial frame (m)} \end{cases}$

2) *Dynamic tire model*: The lateral tire force is approximately linear for small slip angles ($|\alpha| \lesssim 3$), as shown in Figure 2. This linear region assumption is adopted for all subsequent modeling and control design.

$$F_{c,i} = -C_{\alpha,i} \alpha_i, \quad i \in \{f, r\} \quad (3)$$

where $\begin{cases} C_{\alpha,i} & \text{cornering stiffness at axle } i \text{ (N/rad)} \\ \alpha_i & \text{tire slip angle at axle } i \text{ (rad)} \\ i & \text{axle index: } f \text{ (front), } r \text{ (rear)} \end{cases}$

3) Actuator Models:

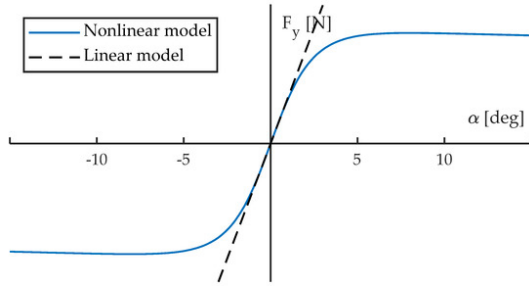


Fig. 2: Linearized dynamic tire model



Fig. 3: 400KV BLDC motor



Fig. 4: 23kg.cm digital RC servo

a) *BLDC Motor*: Each rear motor is driven via an ESC operating in PWM mode. The net tractive force is modeled as a first-order system:

$$\dot{F}_{\text{long}} = \frac{1}{\tau_e} (\eta (V_{\text{batt}} - V_{\text{sag}}) \text{PWM} - F_{\text{long}}) \quad (4)$$

where $\begin{cases} F_{\text{long}} & \text{net longitudinal tractive force (N)} \\ \tau_e & \text{motor electrical time constant (s)} \\ \eta & \text{motor efficiency factor} \\ V_{\text{batt}} & \text{battery voltage (V)} \\ V_{\text{sag}} & \text{voltage sag under load (V)} \\ \text{PWM} & \text{normalised PWM duty cycle input} \in [-1, 1] \end{cases}$

The individual wheel torques and longitudinal forces are given by:

$$J_w \dot{\omega}_i = \tau_{m,i} - r_w F_{x,i}, \quad i \in \{L, R\} \quad (5)$$

where $\begin{cases} J_w & \text{wheel and rotor combined inertia (kg-m}^2\text{)} \\ \tau_{m,i} & \text{motor torque at wheel } i \text{ (N-m)} \\ r_w & \text{wheel radius (m)} \\ F_{x,i} & \text{longitudinal force at wheel } i \text{ (N)} \end{cases}$

The net tractive force and torque-vectoring (TV) yaw moment acting on the vehicle are then

$$F_{\text{long}} = F_{x,L} + F_{x,R}, \quad M_z^{TV} = \frac{t_r}{2} (F_{x,R} - F_{x,L}) \quad (6)$$

where $\begin{cases} t_r & \text{rear track width (m)} \\ M_z^{TV} & \text{torque-vectoring yaw moment (N-m)} \end{cases}$

b) *Steering Servo*.: The RC servo actuating the Ackermann mechanism is modeled as a first-order lag between the commanded and actual steering angle. As the servo has a

reliable inner loop already running, we assume it is well-suited and not under excessive load:

$$\dot{\delta}_f = \frac{1}{\tau_s} (\delta_{\text{ref}} - \delta_f) \quad (7)$$

where $\begin{cases} \tau_s & \text{servo mechanical time constant (s)} \\ \delta_{\text{ref}} & \text{reference steering angle command (rad)} \end{cases}$

4) *Observer Models*: The vehicle is equipped with three sensing modalities that together provide complete observability of the state vector $\mathbf{x}$. The general observation model takes the form

a) *Incremental Encoders*: Each rear wheel is fitted with an incremental encoder measuring wheel angular velocity. The longitudinal velocity and wheel slip ratio are derived as

$$v_x \approx \frac{r_w (\omega_L + \omega_R)}{2}, \quad \kappa_i = \frac{r_w \omega_i - v_x}{v_x}, \quad i \in \{L, R\} \quad (8)$$

where $\begin{cases} r_w & \text{wheel radius (m)} \\ \omega_L, \omega_R & \text{left and right wheel angular velocities (rad/s)} \\ \kappa_i & \text{longitudinal slip ratio at wheel } i \end{cases}$

b) *Inertial Measurement Unit*: The MPU6050 is a 6-DOF MEMS device combining a 3-axis accelerometer and a 3-axis gyroscope. The accelerometer measures linear acceleration along each body-frame axis, while the gyroscope measures angular velocity about each body-frame axis. For the planar 3-DOF model, the relevant outputs are:

$$\mathbf{y}_{\text{IMU}} = \begin{bmatrix} a_{x,\text{meas}} \\ a_{y,\text{meas}} \\ \dot{\psi}_{\text{meas}} \end{bmatrix} = \begin{bmatrix} \dot{v}_x - v_y \dot{\psi} \\ \dot{v}_y + v_x \dot{\psi} \\ \dot{\psi} \end{bmatrix} + \mathbf{n}_{\text{IMU}} \quad (9)$$

where $\begin{cases} a_{x,\text{meas}}, a_{y,\text{meas}} & \text{body-frame linear accelerations from accelerometer (m/s}^2\text{)} \\ \dot{\psi}_{\text{meas}} & \text{yaw angular velocity from gyroscope (rad/s)} \\ \mathbf{n}_{\text{IMU}} & \text{zero-mean Gaussian measurement noise} \end{cases}$

c) *Monocular Camera*: The forward-facing monocular camera is used for perception. The onboard vision pipeline detects the left and right lane boundaries of the track, from which a set of waypoints is extracted to define the reference trajectory with the help of ready-to-use vision pipelines.

$$\mathbf{y}_{\text{cam}} = \begin{bmatrix} y_e \\ \psi_{\text{rel}} \end{bmatrix} + \mathbf{n}_{\text{cam}} \quad (10)$$

where $\begin{cases} y_e & \text{lateral deviation from track centreline (m)} \\ \psi_{\text{rel}} & \text{heading error relative to track tangent (rad)} \\ \mathbf{n}_{\text{cam}} & \text{zero-mean Gaussian measurement noise} \end{cases}$

C. Governing Physical Equations

The dynamics of the vehicle are described by the standard 3-DOF bicycle model. The equations of motion in the fixed body frame are given as follows.

1) Longitudinal Dynamics:

$$m\dot{v}_x = m\dot{\psi}v_y + F_{\text{long}} - F_{\text{drag}} - F_{\text{ramp}} \quad (11)$$

where F_{long} is governed by (4), and the resistive forces are

$$F_{\text{drag}} = k_{\text{drag}} v_x^2, \quad F_{\text{ramp}} = mg \sin \theta \quad (12)$$

where $\begin{cases} k_{\text{drag}} & \text{aerodynamic drag coefficient} \\ g & \text{gravitational acceleration (m/s}^2\text{)} \\ \theta & \text{road grade angle (rad)} \end{cases}$

2) Lateral Dynamics:

$$m\dot{v}_y = -m\dot{\psi}v_x + F_{c,f} \cos(\delta_f) + F_{c,r} \quad (13)$$

3) Yaw Dynamics:

$$I_z \ddot{\psi} = l_f F_{c,f} \cos(\delta_f) - l_r F_{c,r} + M_z^{TV} \quad (14)$$

4) *Tire Force Linearization:* As illustrated in Figure 2, the lateral tire force is approximately linear for small slip angles ($|\alpha| \lesssim 3$). This linear region assumption is adopted for all subsequent modeling and control design:

$$F_{c,i} = -C_{\alpha,i} \alpha_i, \quad i \in \{f, r\} \quad (15)$$

$$\begin{bmatrix} \alpha_f \\ \alpha_r \end{bmatrix} = \begin{bmatrix} \frac{v_y + l_f \dot{\psi}}{v_x} - \delta_f \\ \frac{v_y - l_r \dot{\psi}}{v_x} \end{bmatrix} \quad (16)$$

D. Unified State-Space Representation

Define the state and input vectors as

$$\mathbf{x} = [v_x \quad v_y \quad \dot{\psi}]^T, \quad \mathbf{u} = [F_{x,L} \quad F_{x,R} \quad \delta_f]^T \quad (17)$$

The state-space model is given by

$$\dot{\mathbf{x}} = A(\mathbf{x})\mathbf{x} + B(\mathbf{x})\mathbf{u} + \mathbf{d} \quad (18)$$

where

$$A(\mathbf{x}) = \begin{bmatrix} -\frac{k_{\text{drag}}}{m} & \dot{\psi} & 0 \\ 0 & -\frac{C_{\alpha,f} + C_{\alpha,r}}{mv_x} & \frac{l_r C_{\alpha,r} - l_f C_{\alpha,f}}{mv_x} - v_x \\ 0 & \frac{l_r C_{\alpha,r} - l_f C_{\alpha,f}}{I_z v_x} & -\frac{l_f^2 C_{\alpha,f} + l_r^2 C_{\alpha,r}}{I_z v_x} \end{bmatrix} \quad (19)$$

$$B(\mathbf{x}) = \begin{bmatrix} \frac{1}{m} & \frac{1}{m} & 0 \\ 0 & 0 & \frac{C_{\alpha,f}}{m} \\ -\frac{l_r}{2I_z} & \frac{l_r}{2I_z} & \frac{l_f C_{\alpha,f}}{I_z} \end{bmatrix} \quad (20)$$

where $\begin{cases} F_{x,L}, F_{x,R} & \text{longitudinal forces at rear wheels (N)} \\ \delta_f & \text{front steering angle (rad)} \end{cases}$

V. IDENTIFICATION OF UNCERTAINTIES AND CONTROL CHALLENGES

Autonomous RC racing operates close to the vehicle's handling limits, where small modeling, actuation, and perception errors can rapidly accumulate into loss of tracking or instability. Based on the proposed 3-DOF bicycle model, actuator dynamics, and sensing pipeline, the dominant uncertainties arise from tire-road interaction, actuator imperfections, and measurement noise/latency.

A. Sources of Uncertainty

- **Time-varying tire-road interaction.** In the linear tire region, the lateral forces are modeled as $F_{c,i} = -C_{\alpha,i} \alpha_i$, where the effective cornering stiffness values $C_{\alpha,f}$ and $C_{\alpha,r}$ determine the available lateral grip. In practice, these stiffness values change with surface condition (low-friction patches), speed, and operating point. As a result, the same steering command can generate different yaw and lateral responses at different locations on the track.
- **Operating outside the linear-tire assumption.** The control-oriented model assumes small slip angles, where the force-slip relationship remains approximately linear. However, during aggressive corner entry or a sudden grip reduction, the slip angles can grow beyond this region. When this happens, as shown in Fig. 2, the tire forces become nonlinear and saturate, so the model no longer predicts the vehicle response accurately. Consequently, tracking performance can degrade and oscillations may appear. For this reason, the controller should include saturation handling and safe operating limits.
- **Propulsion uncertainty and battery effects.** The net tractive force is modeled through ESC/motor dynamics and depends on the available battery voltage and voltage sag under load. As the battery discharges, the same PWM command can produce a smaller longitudinal force. Moreover, load variations during acceleration and cornering can increase sag and change the effective force further. This reduces the repeatability of acceleration and makes speed regulation more difficult across a run.
- **Steering actuation lag and saturation.** To capture the fact that the steering cannot change instantaneously, we model the servo as a simple first-order response from the commanded steering angle δ_{ref} to the actual front-wheel steering angle δ_f . This actuator lag becomes more critical at high speed since it can lead to overshoot, oscillatory tracking, and reduces phase margin in the lateral loop. In addition, mechanical limits and servo saturation constrain achievable steering angle and rates, which limits achievable maneuvers in tight curvature.
- **Wheel slip and unmodeled traction effects.** Encoder-based velocity estimates can become unreliable when wheel slip occurs, especially during hard acceleration, braking, or when the vehicle enters low-friction segments. In these cases, the wheels may spin or lock without producing the expected motion on the ground. As a result, the speed inferred from encoder readings may not match the true vehicle longitudinal speed v_x . This mismatch also introduces uncertainty in the effective longitudinal force transmitted to the road, since the same motor command can yield different traction levels under slip.
- **Sensor noise, bias, and perception uncertainty.** The IMU provides accelerations and yaw rate, while the camera pipeline estimates the lateral deviation y_e and relative heading error ψ_{rel} . These measurements are corrupted by noise and can exhibit bias and latency, especially when

the vehicle is vibrating or turning quickly. In addition, the vision pipeline is sensitive to lighting, texture, and motion blur, which can distort lane detection and waypoint extraction. As a result, the controller and the RL policy may receive delayed or inaccurate tracking states, leading to late or incorrect corrective actions.

- **External disturbances and unmodeled dynamics.** Aerodynamic drag and ramp effects act as external disturbances that perturb the longitudinal dynamics and indirectly affect lateral behavior through speed changes. In addition, mechanical compliance, small misalignments, and unmodeled friction also introduce additional dynamics that are not captured in the simplified model. These effects become most disruptive in high-curvature sections, where stability margins are already tight. Therefore, the control strategy must tolerate disturbances and modeling error without losing tracking performance.

B. Resulting Control Challenges

- **Stable path tracking under time-varying grip.** The cornering stiffness C_α directly shapes the lateral and yaw dynamics, so the same steering input can produce very different responses when the surface grip changes. As a result, a fixed-gain controller that is tuned on one operating condition can become poorly tuned in another, leading to larger tracking error or oscillatory behavior. This motivates online estimation of the effective cornering stiffness and an adaptive control strategy that updates the controller based on the current grip level.
- **Speed-dependent dynamics and delay effects at racing conditions.** The lateral response depends strongly on the longitudinal speed v_x , so any speed fluctuation changes how aggressively the car turns for a given steering angle. These speed fluctuations can be introduced by propulsion uncertainty such as battery sag and ESC/motor dynamics. In parallel, camera processing latency, measurement noise, and steering servo lag imply that steering corrections are applied using slightly outdated tracking information. At high speed and in tight curvature, the combination of speed sensitivity and delayed corrections can increase cross-track error and cause overshoot.
- **Estimator-controller interaction and robustness.** Online identification (RLS) must remain numerically stable under sensor noise and wheel slip. Otherwise, the estimated parameters may drift or fluctuate, and the controller may start reacting to estimation noise rather than true dynamics changes. To prevent this, the estimation loop typically requires filtering, projection/bounding of parameter updates, and a carefully chosen update rate. These measures reduce the risk of abrupt parameter transients that could excite the closed-loop system.
- **RL integration without destabilizing the baseline stack.** The RL component should improve local performance while respecting real-time compute limits and safety constraints on the embedded platform. This motivates using compact state features, such as tracking errors

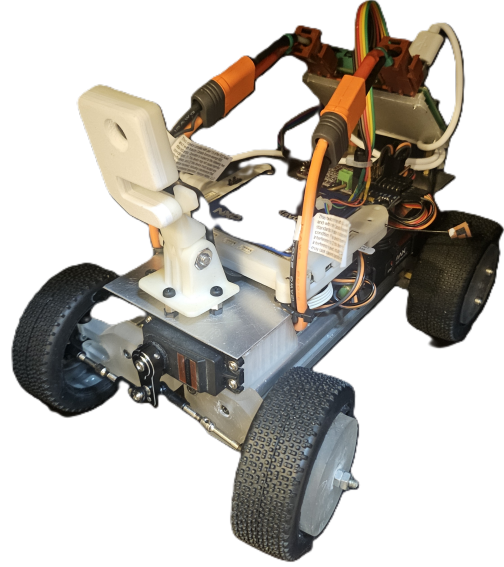


Fig. 5: Physical platform.

and vehicle states, instead of high-dimensional raw sensor inputs. It also motivates bounded actions, such as limited steering or speed adjustments, so that RL refines behavior without overriding the classical waypoint-following pipeline. With this structure, the overall system remains interpretable and easier to validate for stability.

Overall, these uncertainties and challenges motivate a two-pronged approach: (i) adaptive control to handle model mismatch and time-varying tire parameters through online estimation and controller adaptation, and (ii) a constrained RL module to improve performance within the existing waypoint-based architecture while maintaining feasibility for deployment.

VI. PHYSICAL AND SIMULATION PLATFORM

A. Physical Platform

The controllers presented are aimed to be deployed on a physical RC car equipped with a System on Chip (SoC) from NXP, the NavQ Plus running Ubuntu, all the necessary drivers to communicate with the ESCs and the PWM module were developed. The platform is comprised of 2 BLDC motors for propulsion, an RC servo for steering, as well as 2 encoders and a CSI camera for feedback. The main body of the car is made out of a single 20x80 aluminum frame, providing enough sturdiness at a reasonable weight, and ease of mounting the several boards and elements on the car.

Fig5 showcases the finished car in its current state. Fig6 showcases the system block diagram.

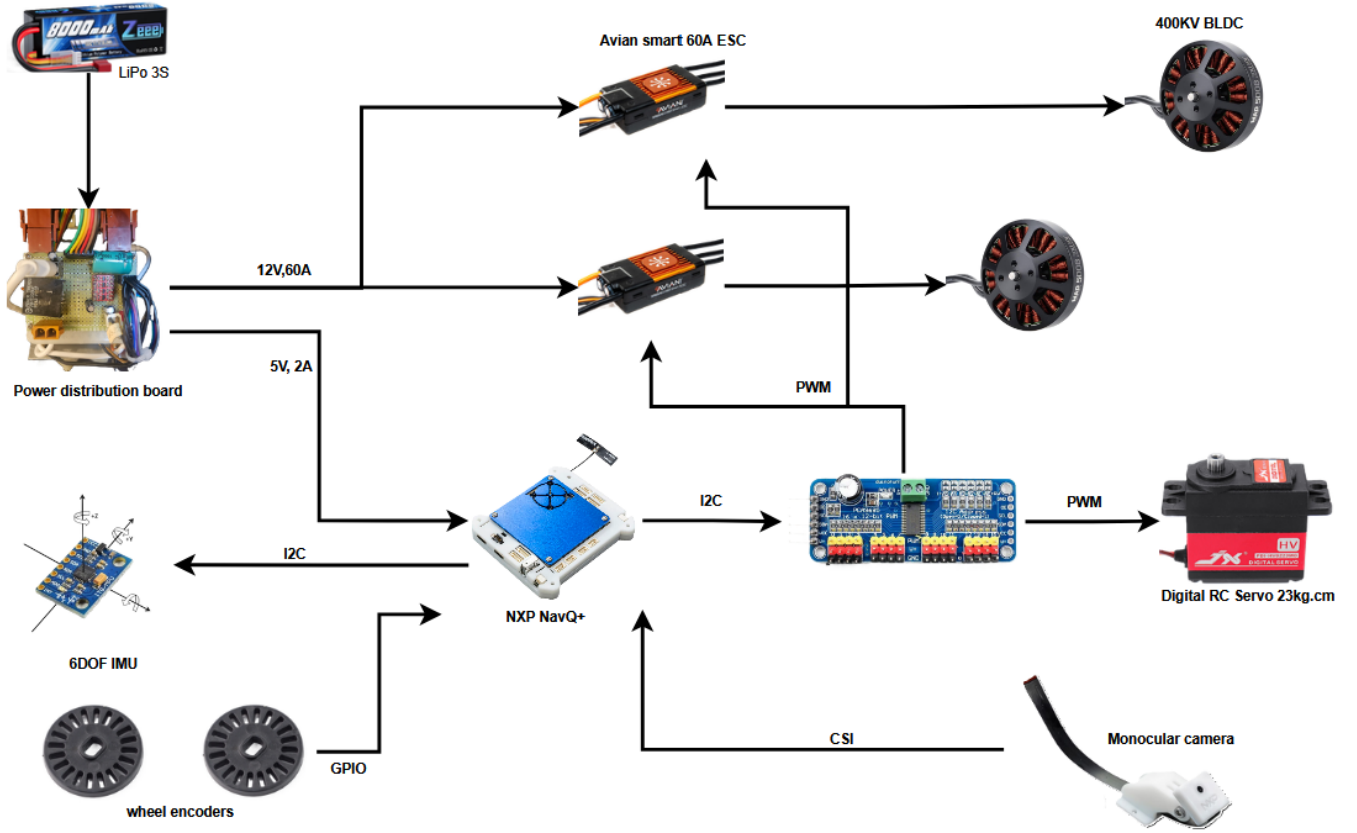


Fig. 6: System block diagram.

B. Simulation Platform

The NXP Cup organizers provide a ready-to-use Gazebo simulation platform, which is used as the base environment for development and testing. Rather than building a simulator from scratch, our approach is to integrate the proposed controllers into the existing ROS-Gazebo framework. To better reflect the challenges targeted in this project, the track has been modified to include a ramp and a tunnel to introduce variations in terrain and perception conditions. This setup provides a practical and efficient environment for evaluating control strategies before hardware implementation. At the current stage, a basic PID controller has been implemented to validate the simulation platform and confirm that the vehicle can be controlled reliably within the modified environment. However, while PID control is sufficient as an initial baseline, its performance is limited in this setting because it depends on fixed gains and does not adapt well to changing operating conditions such as slope transitions, tunnel perception changes, and varying vehicle dynamics along the track. It is therefore used as a baseline for comparison, while adaptive and reinforcement learning based approaches are motivated by their potential to better handle these variations and achieve more robust performance.

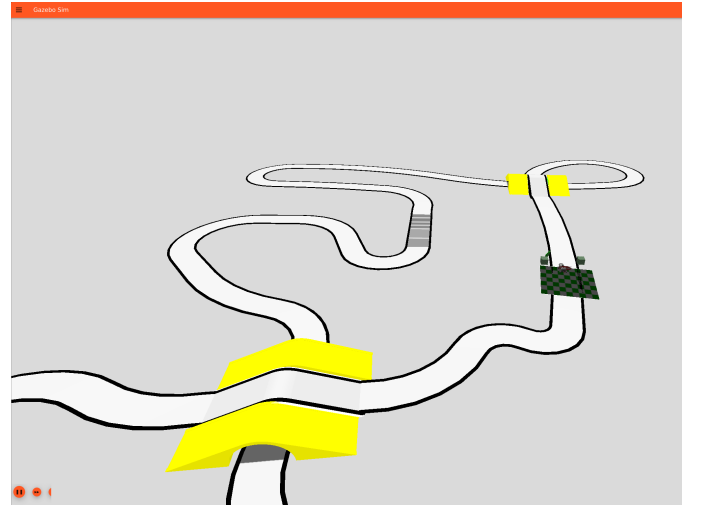


Fig. 7: Gazebo simulation environment with the modified racing track used for controller development and validation.

VII. PROPOSED CONTROLLERS

A. Hierarchical MRAC Design

1) *Architecture Overview:* The full 3-DOF vehicle model exhibits a natural cascade structure that motivates a two-level hierarchical MRAC rather than a single monolithic adaptive

controller. The dominant uncertainties act on well-separated timescales:

- **Outer loop (slow):** battery discharge reduces the effective PWM-to-force gain b_{batt} , affecting longitudinal speed v_x over seconds to minutes.
- **Inner loop (fast):** grip variation changes the cornering stiffnesses $C_{\alpha,f}$ and $C_{\alpha,r}$, affecting lateral–yaw dynamics over tens of milliseconds.

The cascade is treated as one-directional for control design purposes: v_x enters the lateral–yaw subsystem as a measured scheduling variable. The coupling term $mr v_y$ that appears in the longitudinal dynamics (11) does feed into the longitudinal acceleration, but it is treated as a bounded disturbance, which is justified under the moderate lateral slip and speed conditions expected during operation. Fig8 Illustrates the detailed cascaded MRAC Hierarchy dissected below.

2) *Actuator Decomposition:* Although the vehicle has three physical actuators two rear BLDC motors and one front steering servo the lateral–yaw controller uses only two independent control channels. The rear motor forces decompose as

$$F_{\text{long}} = F_{x,L} + F_{x,R}, \quad \Delta F_x = F_{x,R} - F_{x,L}, \quad (21)$$

separating propulsion (common mode) from torque vectoring (differential mode). The common-mode force F_{long} is reserved for longitudinal speed regulation by the outer loop and is excluded from the lateral–yaw subsystem. Individual wheel commands are recovered from controller outputs via

$$F_{x,L} = \frac{F_{\text{long}} - \Delta F_x}{2}, \quad F_{x,R} = \frac{F_{\text{long}} + \Delta F_x}{2}. \quad (22)$$

In this architecture, the steering servo δ_f is the primary actuator for lateral path-following, while ΔF_x provides an auxiliary yaw-moment input that improves yaw tracking and stability under varying tire–road conditions.

3) *Outer Loop: Longitudinal MRAC:*

a) *Plant and uncertainty:* Retaining only the longitudinal dynamics from (11) and absorbing drag and ramp terms into a bounded disturbance d_x , the reduced scalar plant is

$$\dot{v}_x = b_{\text{batt}} \text{PWM} + d_x, \quad (23)$$

where the uncertain input gain captures battery discharge through the BLDC model (4):

$$b_{\text{batt}} \triangleq \frac{\eta(V_{\text{batt}} - V_{\text{sag}})}{m \tau_e}. \quad (24)$$

Since $\eta, V_{\text{batt}}, V_{\text{sag}} > 0$ always, $\text{sign}(b_{\text{batt}}) = +1$.

b) *Reference model:* The desired longitudinal response is a first-order system

$$\dot{v}_{x,m} = -a_m v_{x,m} + a_m v_{x,\text{ref}}, \quad a_m > 0, \quad (25)$$

where $v_{x,\text{ref}}$ is the speed profile from the racing-line planner.

c) *Control law and adaptation:* The adaptive longitudinal control law is

$$\text{PWM} = \hat{k}_{r,x} v_{x,\text{ref}} - \hat{k}_x v_x, \quad (26)$$

with scalar error $e_x \triangleq v_x - v_{x,m}$, regressor $\phi_x \triangleq [-v_x, v_{x,\text{ref}}]^T$, and Lyapunov candidate

$$V_x = \frac{1}{2} p_x e_x^2 + \frac{1}{2} \tilde{\theta}_x^T \Gamma_x^{-1} \tilde{\theta}_x, \quad p_x > 0. \quad (27)$$

Cancelling the cross-term in $\dot{V}_x$ yields the adaptation law

$$\dot{\hat{\theta}}_x = -\Gamma_x \phi_x p_x e_x, \quad \hat{\theta}_x \triangleq \begin{bmatrix} \hat{k}_x \\ \hat{k}_{r,x} \end{bmatrix}. \quad (28)$$

The adaptation rate Γ_x is tuned slower than the inner loop to avoid tracking electrical transients. σ -modification is applied to prevent drift during low-excitation phases such as coasting or braking.

d) *Cross-channel feed-forward:* Because both BLDC motors share the same discharge curve, the same gain drop that affects F_{long} also scales the torque-vectoring authority. The estimated gain corrects the known TV input matrix before it is passed to the inner loop:

$$\hat{B}_{\text{TV}} = \frac{\hat{b}_{\text{batt}}}{b_{\text{batt},0}} B_{\text{TV}}, \quad (29)$$

where $b_{\text{batt},0}$ is the nominal fully-charged gain and B_{TV} is defined in (35). This keeps the inner-loop TV channel calibrated without requiring re-adaptation of $\hat{B}_{\text{TV}}$.

4) *Inner Loop: Lateral–Yaw FSF MRAC:*

a) *State selection:* The inner loop operates on the lateral–yaw subsystem. To avoid a notational collision with the global state vector, the lateral–yaw state is denoted

$$x_\ell \triangleq \begin{bmatrix} v_y \\ r \end{bmatrix}, \quad r \triangleq \dot{\psi}, \quad (30)$$

with two-channel control input

$$u \triangleq \begin{bmatrix} \delta_f \\ \Delta F_x \end{bmatrix}. \quad (31)$$

The measured longitudinal speed v_x , regulated by the outer loop to track $v_{x,m}$, is used as a slowly varying scheduling variable frozen at v_{x0} . Two explicit simplifications are made relative to (19)–(20):

- 1) **Scheduling.** v_x is frozen at v_{x0} ; the coupling term $\dot{\psi} v_y$ in (11) is absorbed into the disturbance d , as noted above.
- 2) **Linearisation.** $\cos \delta_f \approx 1$ and the linear tire model $F_{c,i} = -C_{\alpha,i} \alpha_i$ from (15) are applied, consistent with $|\alpha| \lesssim 3$.

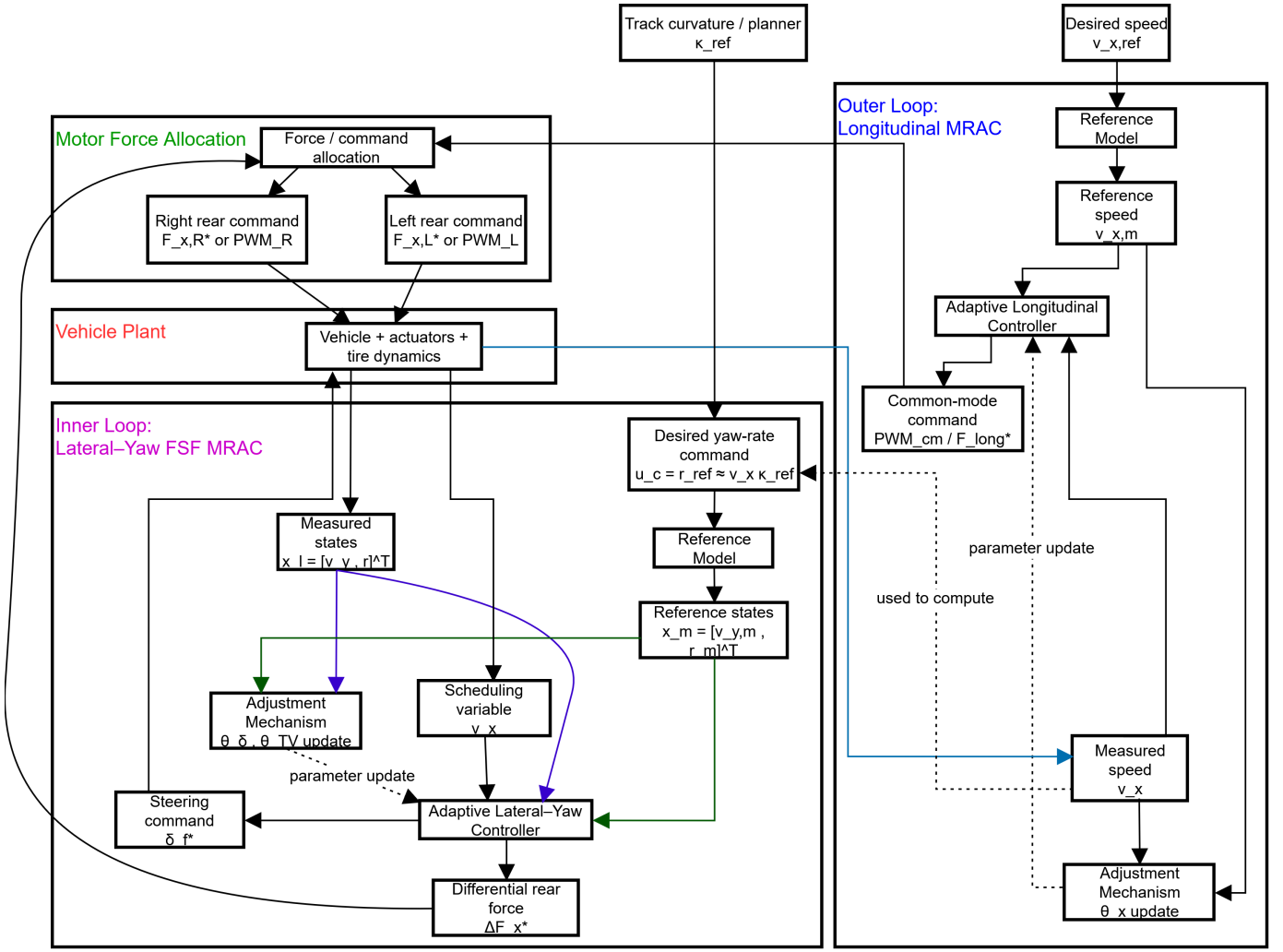


Fig. 8: Cascaded MRAC Hierarchy

b) *Plant model.*: Retaining rows 2–3 of (19) and the corresponding columns of (20), the lateral–yaw dynamics are

$$\dot{x}_\ell = A(\theta) x_\ell + B_\delta(\theta) \delta_f + \hat{B}_{TV} \Delta F_x + d, \quad \theta \triangleq \begin{bmatrix} C_{\alpha,f} \\ C_{\alpha,r} \end{bmatrix}, \quad (32)$$

where

$$A(\theta) = \begin{bmatrix} -\frac{C_{\alpha,f} + C_{\alpha,r}}{mv_{x0}} & -\left(v_{x0} + \frac{l_f C_{\alpha,f} - l_r C_{\alpha,r}}{mv_{x0}}\right) \\ -\frac{l_f C_{\alpha,f} + l_r C_{\alpha,r}}{I_z v_{x0}} & -\frac{l_f^2 C_{\alpha,f} + l_r^2 C_{\alpha,r}}{I_z v_{x0}} \end{bmatrix}. \quad (33)$$

The two input channels are

$$B_\delta(\theta) = B_0 C_{\alpha,f}, \quad B_0 \triangleq \begin{bmatrix} \frac{1}{m} \\ \frac{l_f}{I_z} \end{bmatrix}, \quad \text{sign}(C_{\alpha,f}) = +1, \quad (34)$$

$$B_{TV} = \begin{bmatrix} 0 \\ \frac{t_r}{2I_z} \end{bmatrix}, \quad B_\delta(\theta) = \begin{bmatrix} \frac{C_{\alpha,f}}{m} \\ \frac{l_f C_{\alpha,f}}{I_z} \end{bmatrix}. \quad (35)$$

$B_\delta(\theta)$ is *uncertain*: it depends on $C_{\alpha,f}$, which varies with grip. $\hat{B}_{TV}$ is *known up to the battery correction* (29), as it depends primarily on fixed geometry t_r and I_z . Higher-order effects such as longitudinal tire slip and friction saturation also influence TV effectiveness but are treated as bounded disturbances absorbed into d .

c) *Reference model.*: The reference command is derived from the racing-line curvature:

$$u_c \triangleq r_{\text{ref}} \approx v_{x0} \kappa_{\text{ref}}. \quad (36)$$

The desired closed-loop dynamics are

$$\dot{x}_m = A_m x_m + B_m u_c, \quad (37)$$

with A_m Hurwitz, defined at the nominal high-grip point θ_0 :

$$A_m \triangleq A(\theta_0) - B_\delta(\theta_0) K_{\delta,0} - B_{TV} K_{TV,0}, \quad (38)$$

$$B_m \triangleq B_\delta(\theta_0) k_{r0}, \quad (39)$$

where $K_{\delta,0}$ and $K_{TV,0}$ are obtained by pole placement or LQR over the full input space $[\delta_f, \Delta F_x]$.

d) *Control law.*: Define the shared regressor

$$\phi \triangleq \begin{bmatrix} -v_y \\ -r \\ u_c \end{bmatrix}. \quad (40)$$

The two adaptive FSF control channels are

$$\delta_f = \hat{\theta}_\delta^T \phi = \hat{k}_r u_c - \hat{k}_1 v_y - \hat{k}_2 r, \quad (41)$$

$$\Delta F_x = \hat{\theta}_{TV}^T \phi = \hat{k}_{r,TV} u_c - \hat{k}_3 v_y - \hat{k}_4 r, \quad (42)$$

with adaptive parameter vectors

$$\hat{\theta}_\delta \triangleq \begin{bmatrix} \hat{k}_1 \\ \hat{k}_2 \\ \hat{k}_r \end{bmatrix}, \quad \hat{\theta}_{TV} \triangleq \begin{bmatrix} \hat{k}_3 \\ \hat{k}_4 \\ \hat{k}_{r,TV} \end{bmatrix}. \quad (43)$$

Because $\hat{B}_{TV}$ is known up to the battery correction, the TV channel may alternatively use certainty-equivalence gains from RLS (Section VII-A7), reserving full MRAC adaptation for the uncertain δ_f channel.

5) *Matching Conditions and Error Dynamics.*: Ideal parameters $\theta_\delta^* = [k_1^* \ k_2^* \ k_r^*]^T$ and $\theta_{TV}^* = [k_3^* \ k_4^* \ k_{r,TV}^*]^T$ satisfy the matching conditions. Because $\hat{B}_{TV}$ is used in place of the unknown true TV input matrix, these conditions are written under a certainty-equivalence assumption:

$$A(\theta) - B_\delta(\theta) K_\delta^* - \hat{B}_{TV} K_{TV}^* = A_m, \quad (44)$$

$$B_\delta(\theta) k_r^* + \hat{B}_{TV} k_{r,TV}^* = B_m. \quad (45)$$

Define parameter errors $\tilde{\theta}_\delta = \hat{\theta}_\delta - \theta_\delta^*$ and $\tilde{\theta}_{TV} = \hat{\theta}_{TV} - \theta_{TV}^*$, and the tracking error $e \triangleq x_\ell - x_m$. Subtracting (37) from the closed-loop plant, and applying the factorisation $B_\delta = B_0 b$ with $b \triangleq C_{\alpha,f} > 0$, gives

$$\dot{e} = A_m e + B_0 b \phi^T \tilde{\theta}_\delta + \hat{B}_{TV} \phi^T \tilde{\theta}_{TV}. \quad (46)$$

6) *Lyapunov Design and Adaptation Laws.*: Solve the Lyapunov equation for $P = P^T > 0$:

$$A_m^T P + P A_m = -Q, \quad Q = Q^T > 0. \quad (47)$$

Use the composite Lyapunov candidate

$$V = \frac{1}{2} e^T P e + \frac{1}{2} \tilde{\theta}_\delta^T \Gamma_\delta^{-1} \tilde{\theta}_\delta + \frac{1}{2} \tilde{\theta}_{TV}^T \Gamma_{TV}^{-1} \tilde{\theta}_{TV}, \quad (48)$$

with adaptation gains $\Gamma_\delta, \Gamma_{TV} \succ 0$. Differentiating and substituting (46):

$$\begin{aligned} \dot{V} = & -\frac{1}{2} e^T Q e \\ & + b (B_0^T P e) \phi^T \tilde{\theta}_\delta + (\hat{B}_{TV}^T P e) \phi^T \tilde{\theta}_{TV} \\ & + \tilde{\theta}_\delta^T \Gamma_\delta^{-1} \dot{\tilde{\theta}}_\delta + \tilde{\theta}_{TV}^T \Gamma_{TV}^{-1} \dot{\tilde{\theta}}_{TV}. \end{aligned} \quad (49)$$

Cancelling the cross-terms yields the adaptation laws

$$\dot{\tilde{\theta}}_\delta = -\Gamma_\delta \phi (B_0^T P e), \quad (50)$$

$$\dot{\tilde{\theta}}_{TV} = -\Gamma_{TV} \phi (\hat{B}_{TV}^T P e), \quad (51)$$

where $\text{sign}(b) = +1$ has been used. Substituting back:

$$\dot{V} = -\frac{1}{2} e^T Q e \leq 0, \quad (52)$$

guaranteeing boundedness of all adaptive signals and asymptotic error convergence under standard MRAC assumptions.

a) *Persistent excitation and parameter convergence.*:

The Lyapunov result in (52) guarantees $e \rightarrow 0$ and signal boundedness, but parameter convergence $\tilde{\theta}_\delta, \tilde{\theta}_{TV} \rightarrow 0$ additionally requires the regressor ϕ to be persistently exciting (PE), a condition on the richness of the vehicle's motion rather than the controller. The curvature variation of the racing line provides sufficient excitation under normal cornering. During sustained straight-line running, where $\kappa_{\text{ref}} \approx 0$, PE may be violated; σ -modification then guards against parameter drift until cornering resumes.

7) *RLS Identification of Cornering Stiffness.*:

a) *Regression outputs.*: From the IMU model (9), the directly measurable regression outputs are

$$y_1 \triangleq a_{y,\text{meas}} = \dot{v}_y + v_x r, \quad y_2 \triangleq \dot{r}, \quad (53)$$

where y_1 is read directly from the accelerometer and y_2 is obtained from a filtered derivative of the gyroscope output.

b) *TV correction.*: The full yaw equation (14) includes the torque-vectoring moment $M_z^{TV} = \frac{t_r}{2} \Delta F_x$, so the raw $\dot{r}$ measurement conflates tire-force and TV contributions. The bias-corrected output is

$$y_2^{\text{corr}} \triangleq \dot{r} - \frac{t_r}{2I_z} \Delta F_x. \quad (54)$$

Omitting this correction would systematically bias $\hat{C}_{\alpha,f}$ and $\hat{C}_{\alpha,r}$ whenever $\Delta F_x \neq 0$.

c) *Regression model.*: The corrected output satisfies $y^{\text{corr}} = \Phi \theta$ with $\Phi \triangleq [\phi_1^T; \phi_2^T]$. The regressors, consistent with the no-factor-of-2 convention of (19)–(20), are

$$\phi_1^T = \begin{bmatrix} \delta_f & -\frac{v_y + l_f r}{m v_x} & -\frac{v_y - l_r r}{m v_x} \end{bmatrix}, \quad (55)$$

$$\phi_2^T = \begin{bmatrix} l_f \delta_f & -\frac{l_f (v_y + l_f r)}{I_z v_x} & \frac{l_r (v_y - l_r r)}{I_z v_x} \end{bmatrix}. \quad (56)$$

d) *RLS update.*: At sample k , with forgetting factor $0 < \lambda \leq 1$:

$$K_k = \frac{P_{k-1} \Phi_k^T}{\lambda + \Phi_k P_{k-1} \Phi_k^T}, \quad P_k = \frac{1}{\lambda} (P_{k-1} - K_k \Phi_k P_{k-1}), \quad (57)$$

$$\hat{\theta}_k = \hat{\theta}_{k-1} + K_k (y_k^{\text{corr}} - \Phi_k \hat{\theta}_{k-1}). \quad (58)$$

e) *Certainty-equivalence gains.*: The RLS estimate $\hat{\theta} = [\hat{C}_{\alpha,f} \ \hat{C}_{\alpha,r}]^T$ reconstructs $\hat{A} = A(\hat{\theta})$ from (33) and $\hat{B}_\delta = B_\delta(\hat{\theta})$ from (34). Define the closed-loop matrix

$$\hat{A}_{cl} \triangleq \hat{A} - [\hat{B}_\delta \ \hat{B}_{TV}] \begin{bmatrix} \hat{K}_\delta^T \\ \hat{K}_{TV}^T \end{bmatrix}. \quad (59)$$

Pole placement over the full two-input plant gives

$$[\hat{K}_\delta \ \hat{K}_{TV}] = \text{place}(\hat{A}, [\hat{B}_\delta \ \hat{B}_{TV}], p_m), \quad (60)$$

$$\hat{k}_r = \frac{1}{c_r \hat{A}_{cl}^{-1} \hat{B}_\delta}, \quad c_r \triangleq [0 \ 1], \quad (61)$$

so that the yaw-rate output $r = c_r x_\ell$ tracks u_c . In practice, (60)–(61) initialise $\hat{\theta}_\delta$ and $\hat{\theta}_{TV}$, while the adaptation laws (50)–(51) handle residual mismatch and fast transients. In the RLS channel, PE is equivalently the condition that $\Phi_k^T \Phi_k$ remains sufficiently rich, preventing covariance windup and ensuring $\hat{\theta}_k \rightarrow \theta$. The forgetting factor $\lambda < 1$ further guards against rank deficiency by continuously discounting stale data.

f) *Practical notes.*:

- **Projection bounds.** Enforce $\hat{C}_{\alpha,f}, \hat{C}_{\alpha,r} \in [\theta_{\min}, \theta_{\max}]$ to maintain physical realisability of the estimates.
- **σ -modification.** Append $-\sigma_\delta \hat{\theta}_\delta$ and $-\sigma_{TV} \hat{\theta}_{TV}$ to (50)–(51) to prevent parameter drift under low excitation.
- **Force allocation.** Controller outputs $(\Delta F_x, F_{\text{long}})$ are converted to individual wheel commands via (22), subject to the traction limit $|F_{x,i}| \leq \mu F_{z,i}$ before being sent to the ESCs.

B. Reinforcement Learning Controller

The RL module is used independently of the MRAC controller and serves a different purpose: it learns a *waypoint-following residual policy* that improves local trajectory generation using camera- and accelerometer-based observations, while the waypoint generator provides the global track-completion structure. In this setting, the RL agent learns a compact decision policy that maps the current local racing context to a steering correction that improves progress along the waypoint sequence. The formulation below follows the finite MDP framework of Sutton and Barto [11], uses the classical tabular derivation as a baseline, and then moves to a simpler Monte Carlo policy-gradient method, REINFORCE [12].

1) *Markov Decision Process Formulation.*: The racing problem is modeled as a finite-horizon Markov decision process (MDP),

$$\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle, \quad (62)$$

where $\mathcal{S}$ denotes the state space, $\mathcal{A}$ the action space, $\mathcal{P}(s_{t+1} | s_t, a_t)$ the transition probability, $\mathcal{R}(s_t, a_t)$ the reward function, and $\gamma \in (0, 1)$ the discount factor [?].

To preserve the Markov property with a minimal state representation, we define the RL state using the waypoint-relative quantities extracted from the camera stream together with inertial information:

$$s_t = [e_y(t) \ \theta_e(t) \ v_x(t) \ a_x(t)]^T, \quad (63)$$

where e_y is the cross-track error relative to the next waypoint, θ_e is the heading error, v_x is the longitudinal speed, and a_x is the longitudinal acceleration measured by the accelerometer. This state is sufficient for local waypoint tracking because it summarizes the information needed to decide whether the vehicle should remain on the nominal path or apply a corrective residual action.

The action is the residual steering command,

$$a_t = \delta_{rl}(t) \in [-\delta_{\max}, \delta_{\max}], \quad (64)$$

which is added to the nominal waypoint/controller output. The complete steering command is therefore

$$\delta_{\text{ref}}(t) = \delta_{\text{wp}}(t) + \delta_{rl}(t), \quad (65)$$

where δ_{wp} is the steering command produced by the waypoint generator and/or the nominal path tracker.

2) *Reward Design.*: The reward follows the structure used in autonomous racing reward formulations, where forward progress along the racing line is encouraged while track deviation and excessive control effort are penalized [2], [13]. The reward at time t is defined as

$$\mathcal{R}(s_t, a_t) = \begin{cases} -1, & \text{if collision or track departure occurs,} \\ +1, & \text{if the lap is completed,} \\ \beta_v v_x(t) \cos(\theta_e(t)) - \beta_e |e_y(t)| - \beta_\delta |a_t|, & \text{otherwise} \end{cases} \quad (66)$$

where $\beta_v, \beta_e, \beta_\delta > 0$ are tunable weights. The term $v_x \cos(\theta_e)$ rewards forward motion aligned with the waypoint direction, $|e_y|$ penalizes lateral deviation, and $|a_t|$ regularizes the residual action so that the RL module only intervenes when needed. This reward is compatible with waypoint generation because it rewards track completion indirectly through local progress toward the next waypoint sequence [2].

3) *Discrete-State Baseline.*: To derive the RL formulation from first principles, we first consider a discrete approximation. Let the continuous state variables be quantized into finite bins:

$$\tilde{\mathcal{S}} = \{s^{(1)}, s^{(2)}, \dots, s^{(N)}\}, \quad (67)$$

and let the action set be discretized into

$$\tilde{\mathcal{A}} = \{-\delta_s, 0, +\delta_s\}. \quad (68)$$

This finite formulation is useful because it exposes the fundamental value-based structure of the problem and connects directly to classical dynamic programming and Q-learning [11].

For a policy π , the action-value function is

$$q_\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a], \quad (69)$$

where the return is

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}. \quad (70)$$

The optimal action-value function satisfies the Bellman optimality equation,

$$q^*(s, a) = \sum_{s', r} \mathcal{P}(s', r \mid s, a) \left[r + \gamma \max_{a'} q^*(s', a') \right]. \quad (71)$$

Since the vehicle dynamics and tire-track interaction are nonlinear and unknown, we do not attempt to model $\mathcal{P}$ explicitly. Instead, in the discrete baseline, we may use Q-learning updates of the form [14]

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a') - Q(S_t, A_t) \right]. \quad (72)$$

This discrete derivation is included for completeness, but it is not practical for the racing platform because discretization causes a large memory burden, poor interpolation between neighboring states, and discontinuous action changes.

4) *Continuous Policy Parameterization*: Because the actual control problem is continuous, we replace the tabular representation with a parametric policy. A simple and computationally light choice is a Gaussian stochastic policy,

$$\pi_\theta(a \mid s) = \mathcal{N}(\mu_\theta(s), \sigma^2), \quad (73)$$

where $\mu_\theta(s)$ is a neural network and σ^2 is fixed during training. This avoids the need for a continuous $\arg \max_a Q(s, a)$ search at runtime and keeps the online controller lightweight.

The objective is to maximize the expected discounted return,

$$J(\theta) = \mathbb{E}_{\pi_\theta} \left[\sum_{t=0}^T \gamma^t R_{t+1} \right]. \quad (74)$$

Using the likelihood-ratio identity, the policy-gradient becomes [11], [12]

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta} \left[\sum_{t=0}^T \nabla_\theta \log \pi_\theta(a_t \mid s_t) G_t \right]. \quad (75)$$

For the Gaussian policy,

$$\log \pi_\theta(a_t \mid s_t) = -\frac{1}{2} \log(2\pi\sigma^2) - \frac{(a_t - \mu_\theta(s_t))^2}{2\sigma^2}, \quad (76)$$

and therefore

$$\nabla_\theta \log \pi_\theta(a_t \mid s_t) = \frac{a_t - \mu_\theta(s_t)}{\sigma^2} \nabla_\theta \mu_\theta(s_t). \quad (77)$$

Algorithm 1 REINFORCE-Based RL Controller for Waypoint Tracking

- 1: Initialize policy parameters θ , learning rate α , and discount factor γ
 - 2: Initialize exploration variance σ^2
 - 3: Load waypoint sequence and nominal steering policy δ_{wp}
 - 4: **for** each episode **do**
 - 5: Reset vehicle and observe initial sensor measurements
 - 6: Construct initial state $s_0 = [e_y, \theta_e, v_x, a_x]^\top$
 - 7: **for** each time step t **do**
 - 8: Sample residual action $a_t \sim \pi_\theta(\cdot \mid s_t)$
 - 9: Compute steering command $\delta_{\text{ref}}(t) = \delta_{\text{wp}}(t) + a_t$
 - 10: Apply $\delta_{\text{ref}}(t)$ to the vehicle
 - 11: Observe next state s_{t+1} from camera and accelerometer inputs
 - 12: Compute reward R_{t+1} using the waypoint-based reward function
 - 13: Store (s_t, a_t, R_{t+1})
 - 14: **if** collision or track departure **then**
 - 15: terminate episode
 - 16: **end if**
 - 17: **end for**
 - 18: Compute returns $G_t = \sum_{k=t}^T \gamma^{k-t} R_{k+1}$
 - 19: **for** each stored step t **do**
 - 20: Update policy parameters:

$$\theta \leftarrow \theta + \alpha \gamma^t G_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)$$
 - 21: **end for**
 - 22: **end for**
-

Substituting into the policy-gradient expression yields the REINFORCE update,

$$\theta \leftarrow \theta + \alpha \sum_{t=0}^T \gamma^t G_t \frac{a_t - \mu_\theta(s_t)}{\sigma^2} \nabla_\theta \mu_\theta(s_t), \quad (78)$$

which is a simple Monte Carlo policy-gradient method [12]. Compared with actor-critic methods, REINFORCE is simpler, requires no critic network, and avoids bootstrapped value estimation. This simplicity is desirable here because the waypoint generator already provides a strong structural prior, and the RL module only needs to learn a bounded correction.

5) *Algorithm*: Algorithm 1 summarizes the proposed RL controller. The policy is trained episodically using complete returns, while the waypoint generator provides the local target sequence and the nominal path tracker provides the baseline steering command.

6) *Justification of the Simplified RL Choice*: A policy-gradient method is selected because it works directly with continuous actions and is simpler to compute compared to actor-critic methods. Unlike actor-critic, REINFORCE does not require a learned value function, target networks, or temporal-difference bootstrapping, which reduces algorithmic complexity. At the same time, the policy remains fully compatible with the waypoint-generation pipeline, since the learned

policy only modifies the local steering command rather than replacing the global route structure.

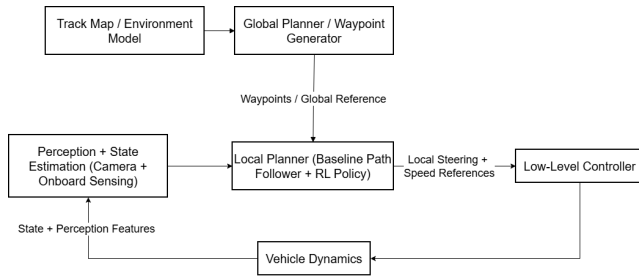


Fig. 9: Top-level control architecture integrating RL within the local planning module.

REFERENCES

- [1] B. Evans, H. A. Engelbrecht, and H. W. Jordaan, "Learning the subsystem of local planning for autonomous racing," in *2021 20th International Conference on Advanced Robotics (ICAR)*, 2021, pp. 601–606.
- [2] B. D. Evans, H. A. Engelbrecht, and H. W. Jordaan, "From navigation to racing: Reward signal design for autonomous racing," *arXiv preprint arXiv:2103.10098*, 2021. [Online]. Available: <https://arxiv.org/abs/2103.10098>
- [3] M. Kim, J.-C. Park, S.-M. Choi, and G.-W. Kim, "Lidar dreamer: Efficient world model for autonomous racing with cartesian-polar encoding and lightweight state-space cells," *Information*, vol. 16, no. 10, p. 898, 2025.
- [4] C.-S. Kim, J.-O. Hahn, K.-S. Hong, and W.-S. Yoo, "Estimation of tire-road friction based on onboard 6-dof acceleration measurement," *IEEE Transactions on Vehicular Technology*, vol. 64, no. 8, pp. 3368–3377, 2015.
- [5] H. Lin, W. Yan, J. Wang, Y. Yao, and B. Gao, "Robust nonlinear speed control for a brushless dc motor using model reference adaptive backstepping approach," in *2009 International Conference on Mechatronics and Automation*, 2009, pp. 335–340.
- [6] A. Melkia, L. Benalia, and Y. Bensalem, "Improved backstepping control strategy for speed regulation of permanent magnet synchronous motors," *Journal of Electrical Systems and Automation*, vol. 58, no. 2, pp. 219–230, 2025.
- [7] MarsvinTech, "Car modelling," Jan. 2021, technical note.
- [8] J. Kong, M. Pfeiffer, G. Schildbach, and F. Borrelli, "Kinematic and dynamic vehicle models for autonomous driving control design," in *Proceedings of the 2015 IEEE Intelligent Vehicles Symposium (IV)*. IEEE, 2015, pp. 1094–1099.
- [9] —, "Kinematic and dynamic vehicle models for autonomous driving control design," in *2015 IEEE Intelligent Vehicles Symposium (IV)*, 2015, pp. 1094–1099.
- [10] The MathWorks, Inc., "Vehicle body 3dof," <https://www.mathworks.com/help/vdynblks/ref/vehiclebody3dof.html>, Natick, Massachusetts, United States, 2024.
- [11] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.
- [12] R. J. Williams, "Simple statistical gradient-following algorithms for connectionist reinforcement learning," *Machine Learning*, vol. 8, no. 3–4, pp. 229–256, 1992.
- [13] Y. Wang, J. Liu, X. Yuan, and J. Yang, "Safe reinforcement learning for competitive autonomous racing: Integrated state-action mapping and exploration guidance framework," *Actuators*, vol. 14, no. 7, 2025. [Online]. Available: <https://www.mdpi.com/2076-0825/14/7/315>
- [14] C. J. C. H. Watkins and P. Dayan, "Q-learning," *Machine Learning*, vol. 8, no. 3–4, pp. 279–292, 1992. [Online]. Available: <https://doi.org/10.1007/BF00992698>